

ADR-0017 - Adopter une architecture microservices pour l'application e-commerce

Statut

Accepted

Date: 2026-03-12

Auteurs: [Nom Architecte], [Nom Lead Tech]

Reviewers: Équipe Architecture, Équipe DevOps, Équipe Backend, Équipe Produit

Contexte et problème

L'application e-commerce actuelle repose sur un monolithe modulaire.

Cette approche a permis un démarrage rapide du produit, une mise en œuvre simple et un coût opérationnel maîtrisé. Cependant, avec la croissance du produit, des équipes et du trafic, plusieurs limites apparaissent.

Nous rencontrons plusieurs problèmes :

1. **Déploiements lourds et risqués** : un changement sur un domaine mineur nécessite le redéploiement de toute l'application
2. **Scalabilité technique limitée** : certaines parties du système, comme le catalogue, les commandes ou le paiement, ont des profils de charge différents mais ne peuvent pas être scalées indépendamment
3. **Couplage croissant entre domaines** : les modules métier restent fortement liés au niveau du runtime, du code ou de la base
4. **Ralentissement des équipes** : plusieurs équipes interviennent sur la même codebase avec conflits fréquents
5. **Résilience insuffisante** : une défaillance locale peut impacter l'ensemble de la plateforme
6. **Contraintes d'évolution technologique** : l'introduction de nouvelles technologies ou stratégies de persistance est difficile à isoler

Le volume actuel est de 20 000 commandes/jour, avec une forte croissance attendue sur le trafic catalogue, les opérations promotionnelles, les intégrations logistiques et les parcours de paiement.

Decision Drivers

- Permettre des déploiements indépendants par domaine
- Améliorer la scalabilité horizontale des composants critiques
- Réduire le couplage opérationnel entre sous-domaines
- Renforcer la résilience globale de la plateforme
- Aligner l'architecture avec l'organisation par équipes métier
- Faciliter l'évolution indépendante de certains domaines
- Préparer l'intégration de nouvelles capacités (marketplace, partenaires externes)

Options considérées

Option 1 : Conserver le monolithe actuel

Continuer à faire évoluer l'application sous forme de monolithe, avec optimisations progressives au niveau du code, du caching et des pipelines CI/CD.

Avantages :

- Pas de transformation architecturale majeure
- Simplicité opérationnelle plus forte
- Développement et débogage centralisés
- Coût d'infrastructure plus faible à court terme

Inconvénients :

- Les déploiements restent globalisés
- Les goulots d'étranglement techniques persistent
- Le couplage entre domaines continue à croître
- La montée en charge ciblée reste limitée
- Les équipes restent contraintes par une codebase partagée

Option 2 : Renforcer le monolithe modulaire

Poursuivre avec une architecture modulaire stricte, avec frontières de modules plus claires, sans séparation en services déployables indépendamment.

Avantages :

- Amélioration de la structure interne
- Complexité opérationnelle plus faible qu'en microservices
- Migration plus progressive
- Bonne étape intermédiaire pour clarifier les frontières métier

Inconvénients :

- Pas de déploiement indépendant réel
- Scalabilité toujours limitée au niveau applicatif global
- Isolation partielle seulement en cas d'incident
- Les contraintes de partage de runtime et d'environnement persistent

Option 3 : Adopter une architecture microservices (choisi)

Découper l'application en services indépendants, chacun responsable d'un domaine métier ou technique cohérent, avec déploiement, scalabilité et cycle de vie propres.

Les premiers services identifiés sont :

- **Catalogue Service**
- **Panier Service**
- **Commandes Service**
- **Paie Service**
- **Stock Service**
- **Livraison Service**
- **Client Service**
- **Promotion Service**

Avantages :

- Déploiements indépendants par service
- Scalabilité ciblée selon la charge métier
- Meilleure isolation des incidents
- Réduction du couplage au niveau du déploiement et du runtime
- Alignement avec l'organisation par équipes orientées domaines
- Liberté accrue d'évolution technologique selon les besoins
- Base adaptée à une croissance forte du produit et des équipes

Inconvénients :

- Complexité opérationnelle nettement plus élevée
- Besoin de gérer la communication distribuée, l'observabilité, la résilience et la sécurité inter-services
- Gestion plus difficile de la cohérence des données
- Débogage et tests end-to-end plus complexes
- Coût infrastructure et DevOps plus élevé
- Risque de mauvais découpage si les frontières métier sont mal définies

Décision

Nous adoptons une architecture microservices pour l'application e-commerce. L'adoption sera **progressive** et guidée par les domaines métier les plus critiques.

L'implémentation reposera sur les principes suivants :

- Découpage des services selon des **frontières métier explicites**
- Chaque microservice possède sa **responsabilité fonctionnelle propre**
- Chaque service peut être **déployé indépendamment**
- Les services exposent des **API contractuelles versionnées**
- Les échanges inter-services privilégient :
 - la communication synchrone pour les cas temps réel strictement nécessaires
 - la communication asynchrone par événements pour les interactions découplées
- La persistance suit le principe **database per service**
- Une plateforme d'observabilité centralisée sera mise en place :
 - logs centralisés
 - métriques
 - traces distribuées

- Un **API Gateway** sera utilisé pour l'exposition externe
- La migration sera effectuée par **extraction progressive** à partir du monolithe existant, en commençant par les domaines les plus adaptés :
 - Catalogue
 - Promotions
 - Stock
 - Commandes

Conséquences

Positives

- Déploiement indépendant des domaines
- Scalabilité plus fine selon le besoin métier
- Meilleure résilience face aux défaillances locales
- Réduction des dépendances fortes dans le runtime
- Meilleur alignement entre architecture et organisation des équipes
- Capacité accrue à faire évoluer rapidement certaines parties critiques
- Préparation plus solide à la croissance fonctionnelle et au trafic

Négatives

- Hausse importante de la complexité distribuée
- Besoin fort en maturité DevOps, SRE et observabilité
- Gestion plus délicate de la cohérence transactionnelle
- Nécessité de mettre en place des patterns de résilience : retry, circuit breaker, idempotence, sagas
- Coût d'exploitation plus élevé
- Courbe d'apprentissage importante pour les développeurs
- Risque d'overengineering si tous les services sont extraits trop tôt

Actions requises

1. Identifier les bounded contexts et valider les frontières de service
2. Définir une stratégie de migration progressive depuis le monolithe
3. Mettre en place l'infrastructure de base :
 - API Gateway
 - observabilité
 - gestion centralisée des secrets
4. Définir les standards de communication inter-services
5. Définir les politiques de versionnement d'API
6. Mettre en place les patterns de résilience et de tolérance aux pannes
7. Extraire progressivement les services Catalogue, Promotions, Stock et Commandes
8. Former les équipes sur les problématiques de systèmes distribués
9. Mesurer les impacts sur la fréquence de déploiement, la stabilité et le time-to-market

Liens

- RFC: </docs/rfc/0013-microservices-adoption-strategy.md>
- Domain Mapping: </docs/architecture/ecommerce-service-boundaries.md>
- Migration Plan: </docs/migration/monolith-to-microservices.md>
- Platform Standards: </docs/platform/microservices-standards.md>